

INTERVIEW SIMULATION SYSTEM

Full Project Report

Based on the provided C++ source code

Prepared for coursework submission

Abstract

This report documents a C++ console-based Interview Simulation System. The system manages interview candidates using a queue data structure implemented through a singly linked list. Candidates are added to the waiting queue, processed in first-in-first-out order, assigned one of five fixed interview time slots, and then moved into a processed candidates list. The report explains the problem background, objectives, system design, data structures, algorithms, program modules, testing considerations, limitations, and possible future improvements.

1. Introduction

Interview scheduling is a common administrative process where candidates must be managed fairly and systematically. A queue is suitable for this type of process because the first candidate who arrives should normally be the first candidate processed. The provided program simulates this process in a menu-driven C++ console application.

The system allows users to add new candidates, assign interview slots, process candidates, view the current queue status, and check the number of candidates still waiting. It also keeps a separate record of candidates who have already been processed.

2. Problem Statement

Manual interview scheduling can become confusing when many candidates are involved. Without a structured process, candidates may be skipped, duplicated, or assigned inconsistently. The program solves this issue by using a queue-based approach where candidate order is preserved and interview slots are assigned during processing.

3. Project Objectives

- To design a simple interview management system using C++.
- To apply the queue concept using a linked-list implementation.
- To store candidate details such as name, ID, age, interview slot, and status.
- To process candidates according to the FIFO principle.
- To display both waiting and processed candidates clearly.
- To demonstrate dynamic memory allocation using pointers and nodes.

4. Scope of the System

The project focuses on a console-based simulation. It does not use a database, graphical user interface, file handling, or online scheduling. All data exists only while the program is running.

5. System Overview

Component	Description
Programming language	C++
Interface type	Console menu interface
Main data structure	Queue implemented using linked list nodes
Queue rule	First-In, First-Out (FIFO)

Main entity	Candidate
Storage during runtime	Dynamic memory using new and delete
Platform dependency	Uses windows.h, Sleep(), and system("cls"), so it is Windows-oriented

6. Data Structure Design

6.1 Candidate Structure

The Candidate structure represents one interview candidate. Each candidate node stores personal information, interview slot information, processing status, and a pointer to the next candidate node.

```
struct Candidate {
    string name;
    int id;
    int age;
    int slot;
    string status;
    Candidate *next;
};
```

Field	Purpose
name	Stores the candidate name.
id	Stores the automatically generated candidate ID.
age	Stores candidate age.
slot	Stores selected interview slot number.
status	Stores candidate status such as Waiting or Processed.
next	Points to the next candidate node in the linked list.

6.2 Queue Class

The Queue class controls the waiting queue and the processed list. It maintains front and rear pointers for the active queue, a processedFront pointer for processed candidates, and numItems to count candidates currently waiting.

```
class Queue {
private:
    Candidate* front;
    Candidate* rear;
    Candidate* processedFront;
    int numItems;
};
```

7. Interview Time Slots

The program provides five fixed interview time slots. The user selects one slot when processing a candidate.

Slot Number	Time
1	9:00 AM - 10:00 AM
2	10:30 AM - 11:30 AM
3	1:00 PM - 2:00 PM
4	2:30 PM - 3:30 PM
5	4:00 PM - 5:00 PM

8. Main Functional Requirements

Menu Option	Function	Explanation
1	Add New Candidate	Accepts candidate name and age, assigns an automatic ID, and adds the candidate to the queue.
2	Process Next Candidate & Assign Slot	Displays the first candidate, asks the user to choose a slot, marks the candidate as processed, and removes the candidate from the waiting queue.
3	View Current Queue	Shows waiting candidates and processed candidates.
4	View Total Candidates	Displays the number of candidates still waiting in the queue.
5	Exit	Terminates the program.

9. Algorithm Explanation

9.1 Add Candidate Algorithm

1. Create a new Candidate object.
2. Read candidate name and age from the user.
3. Assign a unique ID using nextId.
4. Set status to Waiting.
5. Insert the candidate at the rear of the queue.
6. Increase the queue counter.

9.2 Process Candidate Algorithm

7. Check whether the queue is empty.
8. Display the candidate at the front of the queue.
9. Display available interview time slots.
10. Ask the user to select a valid slot between 1 and 5.
11. Store the selected slot in the front candidate node.
12. Dequeue the front candidate.
13. Copy the dequeued candidate into the processed candidates list.
14. Update status to Processed.

9.3 View Queue Algorithm

15. If there are waiting candidates, traverse from front to rear and display their details.
16. If there are processed candidates, traverse the processed list and display their details.
17. If both lists are empty, display that no candidates exist in the system.

10. Program Flow

The overall flow of the application is shown below:

```

Start
|
v
Display Main Menu
|
+-- Option 1: Add new candidate -> Enqueue candidate
|
+-- Option 2: Process candidate -> Assign slot -> Dequeue -> Save to processed list
|
+-- Option 3: View waiting and processed candidates
|
+-- Option 4: Display total waiting candidates
|
+-- Option 5: Exit program
|
Repeat until user selects Exit

```

11. Function and Method Description

Method	Return Type	Purpose
Queue()	Constructor	Initialises all pointers to nullptr and numItems to 0.
~Queue()	Destructor	Clears waiting and processed candidate nodes to prevent memory leaks.
enqueue()	void	Adds a candidate to the rear of the waiting queue.
dequeue()	void	Removes the front candidate and stores a processed copy.
isEmpty()	bool	Checks whether the waiting queue is empty.
clear()	void	Removes all waiting candidates.
assignInterviewSlot()	void	Assigns a slot to the first candidate, although the main program mainly uses direct processing logic.
getTotalInterviewees()	int	Returns the number of candidates currently waiting.
viewCurrentQueue()	void	Displays waiting and processed candidate records.
getFrontCandidate()	Candidate	Returns the first candidate without removing it.
setFrontSlot()	void	Updates the slot value for the first candidate.

12. Input, Processing, and Output

Input	Processing	Output
Candidate name and age	Generate candidate ID and enqueue candidate	Confirmation message and waiting status
Menu choice 2 and selected slot	Validate slot, update candidate slot, dequeue candidate	Processed candidate details and assigned interview time
Menu choice 3	Traverse waiting and processed lists	Full queue status
Menu choice 4	Read numItems counter	Total waiting candidates

13. Sample Scenario

18. User selects option 1 and enters candidate name Ali and age 21.
19. System assigns ID 1001 and stores Ali in the queue.
20. User selects option 1 again and enters candidate name Siti and age 22.
21. System assigns ID 1002 and stores Siti behind Ali.
22. User selects option 2. Ali is processed first because Ali is at the front of the queue.
23. User selects slot 1, so Ali is assigned 9:00 AM - 10:00 AM.
24. Ali is removed from the waiting queue and added to the processed list.
25. Siti becomes the next candidate at the front of the queue.

14. Testing Plan

Test Case	Input / Action	Expected Result	Status
TC01	View queue before adding candidate	System displays no candidates in the system.	Pass if displayed correctly
TC02	Add one candidate	Candidate is added with ID 1001 and status Waiting.	Pass if candidate appears in queue
TC03	Add multiple candidates	Candidates are stored in FIFO order.	Pass if display order is correct
TC04	Process first candidate	First candidate is assigned a slot and moved to processed list.	Pass if waiting count decreases
TC05	Enter invalid slot such as 7	System asks user to enter slot between 1 and 5.	Pass if validation works
TC06	Process while queue is empty	System displays no candidates in queue.	Pass if no crash occurs
TC07	View total candidates	System displays number of waiting candidates only.	Pass if counter is accurate

15. Strengths of the Program

- Uses an appropriate queue structure for interview processing.
- Demonstrates dynamic memory allocation and linked-list traversal.
- Separates waiting candidates from processed candidates.
- Provides a simple menu that is easy to understand.
- Automatically generates candidate IDs to reduce manual input errors.
- Validates interview slot selection to prevent invalid slot values.

16. Limitations

- The program depends on Windows-specific functions such as Sleep() and system("cls").
- Candidate data is lost after the program exits because no file or database storage is used.
- There is no validation for age input, so non-numeric input can cause problems.
- The same time slot can be assigned to multiple candidates because slot availability is not tracked.
- The total candidates option counts only waiting candidates, not all candidates in the system.

- There is no search, edit, or delete function for candidate records.

17. Suggested Improvements

- Add file handling to save and load candidate records.
- Prevent duplicate use of the same interview slot unless multiple interview rooms are supported.
- Add input validation for age and menu selection.
- Replace Windows-only functions with cross-platform alternatives.
- Add search by candidate ID or name.
- Add edit and delete features for candidate information.
- Add a report summary showing total waiting candidates, processed candidates, and assigned slots.
- Use object-oriented separation such as CandidateManager and SlotManager for cleaner design.

18. Conclusion

The Interview Simulation System successfully demonstrates how a queue can be applied to a real-world scheduling scenario. By using a linked-list queue, the system processes candidates according to the FIFO rule, stores essential candidate information, assigns interview slots, and maintains a processed candidate list. Although the system is simple and console-based, it provides a clear foundation for understanding queue operations, dynamic memory management, and structured program design in C++.

Appendix A: Source Code

The following source code was used as the basis for this report.

```
#include<iostream>
#include <windows.h>
using namespace std;

struct Candidate {
    string name;
    int id;
    int age;
    int slot;
    string status;
    Candidate *next;
};

const string TIME_SLOTS[] = {
    "9:00 AM - 10:00 AM", // Slot 1
    "10:30 AM - 11:30 AM", // Slot 2
    "1:00 PM - 2:00 PM", // Slot 3
    "2:30 PM - 3:30 PM", // Slot 4
    "4:00 PM - 5:00 PM" // Slot 5
};

class Queue{
private:
    Candidate* front;
    Candidate* rear;
    Candidate* processedFront;
    int numItems;

public:
    //Constructor
    Queue() : front(nullptr), rear(nullptr), processedFront(nullptr), numItems(0) {}

    //Destructor
    ~Queue() {
        clear();
        while (processedFront != nullptr) {
```

```

        Candidate* temp = processedFront;
        processedFront = processedFront->next;
        delete temp;
    }
}

//Destroy Queue
void enqueue(const Candidate& item){
    Candidate* newNode = new Candidate;
    *newNode = item;
    newNode->next = nullptr;

    if(isEmpty()){
        front = newNode;
        rear = newNode;
    } else{
        rear->next = newNode;
        rear = newNode;
    }
    numItems++;
}

void dequeue(Candidate& item) {
    Candidate* temp = nullptr;

    if (isEmpty()) {
        cout << "The queue is empty.\n";
    } else {
        item = *front;

        Candidate* processedNode = new Candidate;
        *processedNode = item;
        processedNode->status = "Processed";
        processedNode->next = nullptr;

        if (processedFront == nullptr) {
            processedFront = processedNode;
        } else {
            Candidate* current = processedFront;
            while (current->next != nullptr) {
                current = current->next;
            }
            current->next = processedNode;
        }

        temp = front;
        front = front->next;
        delete temp;
        numItems = numItems - 1;
        if(front == nullptr) {
            rear = nullptr;
        }
    }
}

bool isEmpty() const{
    return numItems == 0;
}

void clear(){
    Candidate candidate;
    while (!isEmpty()){
        dequeue(candidate);
    }
}

void assignInterviewSlot() {
    if (isEmpty()) {
        cout << "No candidates in the queue to assign an interview slot.\n";
    } else {
        cout << "\nAssigning slot for the first candidate in queue:\n";
        cout << "-----\n";
        cout << "Candidate Name: " << front->name << "\n";
        cout << "Candidate ID: " << front->id << "\n";
        cout << "\nAvailable Time Slots:\n";
    }
}

```

```

    for (int i = 0; i < 5; i++) {
        cout << i + 1 << ". " << TIME_SLOTS[i] << "\n";
    }

    int selectedSlot;
    do {
        cout << "\nSelect interview slot (1-5): ";
        cin >> selectedSlot;
        if (selectedSlot < 1 || selectedSlot > 5) {
            cout << "Invalid slot! Please select between 1-5.\n";
        }
    } while (selectedSlot < 1 || selectedSlot > 5);

    front->slot = selectedSlot;
    cout << "\nSlot assigned successfully!\n";
    cout << "Candidate: " << front->name << "\n";
    cout << "Assigned Time: " << TIME_SLOTS[selectedSlot - 1] << "\n";
    cout << "\nProcessing...";
    Sleep(3000);
    system("cls");
}

int getTotalInterviewees() const {
    return numItems;
}

void viewCurrentQueue() const {
    if (!isEmpty()) {
        cout << "\nWAITING CANDIDATES:\n";
        cout << "-----\n";
        Candidate* current = front;
        while (current != nullptr) {
            cout << "Name: " << current->name << "\n";
            cout << "ID: " << current->id << "\n";
            cout << "Age: " << current->age << "\n";
            cout << "Time Slot: Waiting to be processed" << endl;
            cout << "Status: Waiting\n";
            cout << "-----\n";
            current = current->next;
        }
    }

    if (processedFront != nullptr) {
        cout << "\nPROCESSED CANDIDATES:\n";
        cout << "-----\n";
        Candidate* current = processedFront;
        while (current != nullptr) {
            cout << "Name: " << current->name << "\n";
            cout << "ID: " << current->id << "\n";
            cout << "Age: " << current->age << "\n";
            cout << "Time Slot: ";
            if (current->slot > 0 && current->slot <= 5) {
                cout << TIME_SLOTS[current->slot - 1];
            } else {
                cout << "Waiting to be processed";
            }
            cout << endl;
            cout << "Status: " << current->status << "\n";
            cout << "-----\n";
            current = current->next;
        }
    }

    if (isEmpty() && processedFront == nullptr) {
        cout << "\nNo candidates in the system.\n";
    }
}

Candidate getFrontCandidate() const {
    if (!isEmpty()) {
        return *front;
    }
    throw runtime_error("Queue is empty");
}

```

```

void setFrontSlot(int slot) {
    if (!isEmpty() && slot >= 1 && slot <= 5) {
        front->slot = slot;
    }
}

};

int main() {
    Queue interviewQueue;
    int choice;
    int nextId = 1001;

    do {
        system("cls");
        cout << "\n=== Interview Simulation System ===\n";
        cout << "1. Add New Candidate\n";
        cout << "2. Process Next Candidate & Assign Slot\n";
        cout << "3. View Current Queue\n";
        cout << "4. View Total Candidates\n";
        cout << "5. Exit\n";
        cout << "\nEnter your choice (1-5): ";
        cin >> choice;

        switch (choice) {
            case 1: {
                Candidate newCandidate;
                cin.ignore();

                cout << "\nEnter candidate name: ";
                getline(cin, newCandidate.name);

                newCandidate.id = nextId++;
                cout << "\nAssigned ID: " << newCandidate.id << endl;

                cout << "\nEnter candidate age: ";
                cin >> newCandidate.age;

                newCandidate.status = "Waiting";
                newCandidate.next = nullptr;

                interviewQueue.enqueue(newCandidate);
                cout << "\nCandidate added successfully!\n";
                cout << "Time Slot: Waiting to be processing\n";
                cout << "\nProcessing application...";
                Sleep(3000);
                system("cls");
                break;
            }

            case 2: {
                Candidate processedCandidate;
                if (!interviewQueue.isEmpty()) {
                    try {
                        Candidate frontCandidate = interviewQueue.getFrontCandidate();
                        cout << "\nProcessing next candidate:\n";
                        cout << "-----\n";
                        cout << "ID: " << frontCandidate.id << endl;
                        cout << "Name: " << frontCandidate.name << endl;
                        cout << "Age: " << frontCandidate.age << endl;
                        cout << "-----\n";

                        cout << "\nAvailable Time Slots:\n";
                        for (int i = 0; i < 5; i++) {
                            cout << i + 1 << ". " << TIME_SLOTS[i] << "\n";
                        }

                        int selectedSlot;
                        do {
                            cout << "\nSelect interview slot (1-5): ";
                            cin >> selectedSlot;
                            if (selectedSlot < 1 || selectedSlot > 5) {
                                cout << "Invalid slot! Please select between 1-5.\n";
                            }
                        } while (selectedSlot < 1 || selectedSlot > 5);
                    }
                }
            }
        }
    } while (choice != 5);
}

```

```

        interviewQueue.setFrontSlot(selectedSlot);

        interviewQueue.dequeue(processedCandidate);

        cout << "\nCandidate processed successfully!\n";
        cout << "Assigned Time: " << TIME_SLOTS[selectedSlot - 1] << "\n";
        cout << "\nProcessing...";
        Sleep(3000);
    } catch (runtime_error& e) {
        cout << e.what() << endl;
        Sleep(2000);
    }
} else {
    cout << "\nNo candidates in the queue to process.\n";
    Sleep(2000);
}
system("cls");
break;
}

case 3: {
    system("cls");
    cout << "\n=== Interview Queue Status ===\n";
    interviewQueue.viewCurrentQueue();
    cout << "\nPress Enter to return to main menu...";
    cin.ignore();
    cin.get();
    system("cls");
    break;
}

case 4: {
    system("cls");
    cout << "Total candidates in queue: "
        << interviewQueue.getTotalInterviewees() << endl;
    cout << "\nPress Enter to return to main menu...";
    cin.ignore();
    cin.get();
    system("cls");
    break;
}

case 5: {
    system("cls");
    cout << "\n\nExiting the Interview Simulation System... Goodbye!\n\n";
    break;
}

default: {
    cout << "Invalid choice! Please try again.\n";
    system("cls");
    break;
}
}
} while (choice != 5);

return 0;
}

```